

Week 16: Practice Assignment - Solution

Part 1: DBN for Feature Extraction

```
# Load MNIST dataset and preprocess
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

digits = load_digits()
X = digits.data
y = digits.target

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Build and train DBN (stacked RBMs)
from sklearn.neural_network import BernoulliRBM

rbm1 = BernoulliRBM(n_components=128, learning_rate=0.01, n_iter=10, random_state=42)
X_train_rbm1 = rbm1.fit_transform(X_train)

rbm2 = BernoulliRBM(n_components=64, learning_rate=0.01, n_iter=10, random_state=42)
X_train_rbm2 = rbm2.fit_transform(X_train_rbm1)

# Train classifier on extracted features
classifier = LogisticRegression(max_iter=500)
classifier.fit(X_train_rbm2, y_train)

# Evaluate on test set
X_test_rbm1 = rbm1.transform(X_test)
X_test_rbm2 = rbm2.transform(X_test_rbm1)
y_pred = classifier.predict(X_test_rbm2)

print(f"DBN Classification Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

Part 2: RBM for Recommendation System

```
import numpy as np

# Create synthetic user-item interaction matrix
np.random.seed(42)
num_users, num_items = 100, 50
interaction_matrix = np.random.randint(0, 2, (num_users, num_items))
```

```
# Split data into train and test sets
train_data = interaction_matrix[:80]
test_data = interaction_matrix[80:]

# Build and train RBM
from sklearn.neural_network import BernoulliRBM

rbm = BernoulliRBM(n_components=64, learning_rate=0.01, n_iter=10, random_state=42)
rbm.fit(train_data)

# Predict interactions for a test user
test_user = test_data[0].reshape(1, -1)
predicted_interactions = rbm.transform(test_user)

# Recommend top-N items
recommended_items = np.argsort(-predicted_interactions[0])[0:5]
print(f'Recommended items for test user: {recommended_items}')
```

For learner use only